



UNIVERSITY OF SCIENCE, VNU-HCM

Faculty of Information Technology

CSC10007 - OPERATING SYSTEM

PROJECT 2 - Page Tables and File System

Instructor: Phan Quốc Kỳ

Group Members

Nguyễn Khánh Toàn	MSSV: 24127252
Nguyễn Tiến Cường	MSSV: 24127337
Trần Lưu Gia Bảo	MSSV: 24127018

Semester II (2025 - 2026)

Mục lục

I. GIỚI THIỆU	2
1.1 Thành viên nhóm và phân công công việc	2
1.2 Tổng quan nội dung project	2
II. PHẦN 1: PAGE TABLES	2
2.1 Speed up system call (getpid)	2
2.1.1 Cách cài đặt: ánh xạ trang dùng chung tại USYSCALL .	2
2.1.2 Thiết lập quyền truy cập: chọn bit chỉ đọc cho userspace	3
2.1.3 Phân tích: các system call khác có thể tối ưu theo cách này	3
2.2 Print a page table (vmprint)	4
2.2.1 Tổng quan	4
2.2.2 Triển khai	4
2.2.3 Kiểm thử	5
2.2.4 Kết quả	5
2.2.5 Kết luận	8

I. GIỚI THIỆU

1.1 Thành viên nhóm và phân công công việc

Nhóm thực hiện Project 2 gồm 3 thành viên với phân công như sau:

STT	Họ và tên	MSSV	Phân công
1	Nguyễn Khánh Toàn	24127252	Thực hiện bài <i>Speed up system calls</i> và chỉnh sửa báo cáo
2	Nguyễn Tiến Cường	24127337	Thực hiện bài <i>Print a page table</i>
3	Trần Lưu Gia Bảo	24127018	Thực hiện bài <i>Large file</i>

1.2 Tổng quan nội dung project

Theo yêu cầu trong nội dung đề án, project gồm hai phần chính:

- **Page Tables:** tối ưu lời gọi hệ thống `getpid()` bằng trang chia sẻ `USYSCALL` và cài đặt hàm `vmprint()` để in cây bảng trang.
- **File System:** mở rộng hệ thống file của xv6 để hỗ trợ file lớn bằng cách thêm cơ chế khối gián tiếp cấp hai (doubly-indirect block).

Mục tiêu của báo cáo là trình bày cách cài đặt, lựa chọn thiết kế, kết quả kiểm thử, và các vấn đề phát sinh trong quá trình thực hiện.

II. PHẦN 1: PAGE TABLES

2.1 Speed up system call (getpid)

2.1.1 Cách cài đặt: ánh xạ trang dùng chung tại USYSCALL

Nhóm tối ưu `getpid()` bằng cách ánh xạ trang dùng chung `USYSCALL` (`USYSCALL = TRAPFRAME - PGSIZE`) để user đọc PID trực tiếp, không cần trap vào kernel.

Các thay đổi chính:

- `proc.h`: thêm `struct usyscall *usyscall` vào `struct proc`.
- `allocproc()`: cấp phát trang và gán `p->usyscall->pid = p->pid`.
- `proc_pagetable()`: map trang vào địa chỉ `USYSCALL`.

- `proc_freepagetable()` và `freeproc()`: unmap và giải phóng trang tương ứng.

Kết quả: `ugetpid()` đọc PID trực tiếp từ `USYSCALL` và `pgtbltest: ugetpid` đạt OK.

```
nguyentoan@nguyentoan-Virtual-Platform:~/Downloads/LAB2_OS/xv6-labs-2024$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if
-none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x1000 pte 0x21FC701B pa 0x87F1C000 perm 0x1B
va 0x2000 pte 0x21FC6C17 pa 0x87F19000 perm 0x17
va 0x3000 pte 0x21FC6807 pa 0x87F1A000 perm 0x7
va 0x4000 pte 0x21FC6407 pa 0x87F19000 perm 0x07
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFD000 pte 0x21FD0013 pa 0x87F40000 perm 0x13
va 0xFFFFE000 pte 0x21FD4C7 pa 0x87F53000 perm 0xC7
va 0xFFFFF000 pte 0x20001840 pa 0x80000000 perm 0x48
print_pgtbl: OK
ugetpid test starting
ugetpid test: OK
print_kpgtbl starting
print_kpgtbl: OK
superpg test starting
pgtbltest: superpg test failed: pte different, pid=3
```

Hình 1: Ảnh minh chứng chạy `pgtbltest: ugetpid`.

2.1.2 Thiết lập quyền truy cập: chọn bit chỉ đọc cho userspace

Trang `USYSCALL` được map với quyền `PTE_R` | `PTE_U`:

- User chỉ đọc được PID.
- User không thể ghi (không có `PTE_W`).
- Trang không thực thi (không có `PTE_X`).

Thiết kế này đảm bảo vừa nhanh vừa an toàn cho dữ liệu PID.

2.1.3 Phân tích: các system call khác có thể tối ưu theo cách này

Cơ chế này phù hợp với các system call trả về dữ liệu nhỏ, chỉ đọc, và được gọi thường xuyên. Ví dụ, `uptime()` có thể được tối ưu bằng cách lưu giá trị `ticks` hiện tại vào trang `USYSCALL` — nhân cập nhật giá trị này mỗi khi có timer interrupt, còn user đọc trực tiếp mà không cần trap vào kernel.

Với các system call có tác dụng phụ hoặc yêu cầu kiểm tra đặc quyền, cách này không áp dụng được.

2.2 Print a page table (vmprint)

2.2.1 Tổng quan

Bài tập yêu cầu cài đặt hàm `vmprint()` trong nhân xv6 để duyệt và in ra cấu trúc bảng trang ba cấp của một tiến trình theo sơ đồ phân cấp. Hàm này được gọi ngay trước khi `exec()` trả về đối với tiến trình `init` (PID 1), tức là tiến trình người dùng đầu tiên được khởi chạy khi hệ điều hành khởi động.

Mục tiêu cụ thể là:

- Hiểu cơ chế phân trang Sv39 của RISC-V — mỗi địa chỉ ảo 39-bit được chia thành ba chỉ số VPN (mỗi chỉ số 9-bit) và một offset 12-bit, tạo thành cây bảng trang ba tầng, mỗi tầng chứa 512 mục (PTE).
- Phân biệt PTE lá (có cờ R/W/X) với PTE nội bộ (trỏ đến bảng trang tầng dưới, không có R/W/X) theo đặc tả kiến trúc RISC-V.
- In đúng định dạng thật lè theo tầng và chỉ in các PTE hợp lệ (bit `PTE_V` được bật).

2.2.2 Triển khai

Để hoàn thành bài tập, ba thay đổi được thực hiện trong mã nguồn nhân xv6:

Hàm duyệt bảng trang (`kernel/vm.c`).

Một hàm đệ quy nội bộ được thêm vào để duyệt qua toàn bộ 512 mục của một bảng trang ở một tầng nhất định. Với mỗi PTE hợp lệ, hàm in ra chỉ số, giá trị thô của PTE và địa chỉ vật lý tương ứng, kèm theo thật lè phù hợp với độ sâu hiện tại trong cây (tầng 0 in "...", tầng 1 in "... ..", tầng 2 in "... .. .").

Sau khi in, nếu PTE đó *không* mang cờ đọc/ghi/thực thi, tức là nó đang trỏ đến một bảng trang cấp dưới, hàm tự gọi đệ quy vào bảng trang đó với độ sâu tăng thêm một. Cách phân biệt PTE lá và PTE nội bộ này tuân đúng đặc tả kiến trúc RISC-V và nhất quán với hàm `freewalk()` đã có trong `vm.c`.

Hàm công khai `vmprint()` chỉ đơn giản in dòng tiêu đề chứa địa chỉ vật lý của bảng trang gốc rồi khởi động quá trình đệ quy từ tầng 0.

Điểm gọi trong `kernel/exec.c`.

Một lệnh gọi `vmprint()` được chèn vào cuối hàm `exec()`, ngay trước câu

lệnh trả về, có điều kiện chỉ chạy khi tiến trình hiện tại có PID bằng 1. Nhờ điều kiện này, đầu ra chỉ xuất hiện đúng một lần khi `init` được nạp, không ảnh hưởng đến bất kỳ tiến trình nào khác.

Khai báo nguyên mẫu trong `kernel/defs.h`.

Nguyên mẫu hàm `vmprint()` được khai báo trong tệp tiêu đề dùng chung `defs.h` để `exec.c` có thể tham chiếu mà không cần include thêm tệp nào khác. Cả ba thay đổi trên đều được bao trong chỉ thị biên dịch có điều kiện `#ifndef LAB_PGTBL` để không ảnh hưởng đến các cấu hình nhân khác.

2.2.3 Kiểm thử

Sau khi hoàn thành cài đặt, bài kiểm thử được thực hiện bằng cách chạy lệnh `make qemu` trong thư mục gốc của `xv6-labs-2024` trên máy Linux. Tiến trình kiểm thử gồm các bước:

1. Biên dịch toàn bộ nhân `xv6` bằng `make qemu`. Nếu không có lỗi biên dịch, QEMU khởi động mô phỏng máy RISC-V.
2. Quan sát đầu ra trên màn hình terminal ngay sau dòng “*xv6 kernel is booting*”. Đầu ra của `vmprint()` xuất hiện trước dòng “*init: starting sh*”, tức là trước khi shell người dùng khởi chạy.
3. Kiểm tra cấu trúc cây bảng trang in ra có đúng định dạng thật lè, đúng số tầng và đúng số lượng mục hợp lệ theo lý thuyết hay không.
4. Chạy bộ kiểm thử tự động `pgtbltest` trong shell `xv6` và xác nhận bằng lệnh `./grade-lab-pgtbl pte_printout` để chắc chắn đầu ra khớp với kết quả mong đợi trong bộ chấm điểm.

2.2.4 Kết quả

Hình 2 dưới đây là ảnh chụp màn hình terminal thực tế khi chạy `make qemu`, cho thấy đầu ra của `vmprint()` xuất hiện đúng vị trí và đúng định dạng.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● cuong@cuong-VirtualBox:/media/sf_os-project/project_2$ cd xv6-labs-2024
○ cuong@cuong-VirtualBox:/media/sf_os-project/project_2/xv6-labs-2024$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -no
graphic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,
id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
page table 0x0000000087f4d000
..0: pte 0x0000000021fd2401 pa 0x0000000087f49000
.. ..0: pte 0x0000000021fd2001 pa 0x0000000087f48000
.. .. ..0: pte 0x0000000021fd281b pa 0x0000000087f4a000
.. .. ..1: pte 0x0000000021fd1c17 pa 0x0000000087f47000
.. .. ..2: pte 0x0000000021fd1807 pa 0x0000000087f46000
.. .. ..3: pte 0x0000000021fd1417 pa 0x0000000087f45000
..255: pte 0x0000000021fd3001 pa 0x0000000087f4c000
.. ..511: pte 0x0000000021fd2c01 pa 0x0000000087f4b000
.. .. ..509: pte 0x0000000021fd5813 pa 0x0000000087f56000
.. .. ..510: pte 0x0000000021fd5407 pa 0x0000000087f55000
.. .. ..511: pte 0x000000002000180b pa 0x0000000080006000
init: starting sh
$
```

Hình 2: Đầu ra của `vmprint()` khi xv6 khởi động — terminal chạy trên Linux trong môi trường QEMU/RISC-V.

Kết quả in ra cho thấy bảng trang của tiến trình `init` sử dụng đúng hai mục tầng cao nhất: mục số 0 ánh xạ vùng địa chỉ ảo thấp (chứa đoạn text, data, guard page, stack của chương trình `init`) và mục số 255 ánh xạ vùng địa chỉ ảo cao (chứa các trang do nhân cung cấp: `USYSCALL`, `TRAPFRAME`, `TRAMPOLINE`).

Bảng 1 và Bảng 2 tóm tắt ý nghĩa của từng trang lá được in.

Địa chỉ ảo	Cờ quyền	R/W/X/U	Nội dung
0x0000	0x1b	R-XU	Đoạn text — mã máy của <code>init</code> ; có thể đọc và thực thi từ chế độ người dùng, không ghi được.
0x1000	0x17	RW-U	Đoạn data/BSS — biến toàn cục và tĩnh; đọc/ghi được từ chế độ người dùng, không thực thi được.
0x2000	0x07	RW-	Guard page — trang canh stack; không có cờ <code>PTE_U</code> nên mọi truy cập từ người dùng đều kích hoạt lỗi trang.
0x3000	0x17	RW-U	User stack — ngăn xếp ban đầu của <code>main()</code> ; đọc/ghi được từ chế độ người dùng.

Bảng 1: Các trang lá vùng địa chỉ thấp (mục tầng 0, tầng 1 chỉ số 0)

Trang	Cờ quyền	Nội dung
USYSCALL	R-U	Trang chỉ đọc dùng chung giữa nhân và người dùng; lưu PID tiến trình để người dùng đọc nhanh mà không cần system call.
TRAPFRAME	RW-	Khung bẫy — lưu toàn bộ thanh ghi CPU khi có trap; chỉ nhân mới truy cập được.
TRAMPOLINE	R-X-	Trang mã trampoline — thực hiện chuyển đổi không gian địa chỉ khi vào/ra trap; ánh xạ ở cùng địa chỉ ảo trong mọi tiến trình và trong nhân; không cho người dùng thực thi trực tiếp.

Bảng 2: Các trang lá vùng địa chỉ cao (mục tầng 0 chỉ số 255, tầng 1 chỉ số 511)

2.2.5 Kết luận

Bài tập đã được hoàn thành đúng yêu cầu. Hàm `vmprint()` duyệt và in đúng cấu trúc bảng trang ba tầng Sv39, hiển thị thụt lề theo cấp và chỉ in các mục hợp lệ, kết quả khớp hoàn toàn với đầu ra mong đợi của bộ chấm điểm.

Qua bài tập này, một số điểm quan trọng được củng cố:

- **Cơ chế phân trang Sv39:** Mỗi địa chỉ ảo được dịch qua ba tầng bảng trang; mỗi tầng chiếm 9 bit của địa chỉ ảo, mỗi bảng trang chiếm đúng một trang vật lý ($4\text{ KiB} = 512 \text{ mục} \times 8 \text{ byte/mục}$).
- **Phân loại PTE:** PTE không mang cờ R/W/X là con trỏ đến bảng trang cấp dưới; PTE có ít nhất một trong ba cờ đó là trang lá ánh xạ bộ nhớ thực.
- **Bố cục địa chỉ ảo của tiến trình xv6:** Vùng thấp chứa đoạn text, data, guard page và stack của chương trình; vùng cao chứa các trang nhân (TRAPFRAME, TRAMPOLINE) và trang dùng chung (USYSCALL). Sự phân tách giữa trang có thể thực thi và trang có thể ghi (nguyên tắc $W \oplus X$) ngăn ngừa một lớp lớn tấn công chèn mã.